Atelier 3 - Structuration Pierre-Feuille-Ciseaux-Lézard-Spock

En conservant le binôme de l'atelier précédent, vous partez de la correction et organisez le code en fonctions.

Toute syntaxe non vue en cours est considérée hors sujet.

Étape 1 - Création du projet

Créer un projet atelier3 sur github et remplacer le code existant par le code ci-dessous puis le lancer :

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

int main()
{
    int scoreJoueur = 0;
    int scoreOrdi = 0;
    int manche = 1;
    int choixJoueur;
    int choixOrdi;

    printf("=== PIERRE, FEUILLE, CISEAUX, LEZARD, SPOCK (7 Manches / avantage
décisif de 2) ===\n");
    while (manche <= 7
        && scoreJoueur-scoreOrdi < 2
        && scoreOrdi-scoreJoueur < 2)
    {
        printf("--- Manche %d/7 ---\n", manche);

        // Saisie du joueur
        bool incorrect;

        do
        {
            printf("Choix (1 = Pierre, 2 = Feuille, 3 = Ciseaux, 4 = Lézard, 5 =
Spock) :");
            scanf("%d", &choixJoueur);
            incorrect = choixJoueur < 1 || 5 < choixJoueur;
            if(incorrect) {
                printf("Non valide, valeurs de 1 à 5 acceptées\n");
            }
        } while (incorrect);

        // Choix aléatoire de l'ordinateur (1, 2, 3, 4 ou 5)
        choixOrdi = (rand() % 5) + 1;
        printf("L'ordinateur a choisi : %d\n", choixOrdi);

        // Détermination du gagnant de la manche
```

```
        if (choixJoueur == choixOrdi)
        {
            printf("Égalité !\n");
        }
        else if ((choixJoueur == 1 && (choixOrdi == 3 || choixOrdi == 4)) ||
                 (choixJoueur == 2 && (choixOrdi == 1 || choixOrdi == 5)) ||
                 (choixJoueur == 3 && (choixOrdi == 2 || choixOrdi == 4)) ||
                 (choixJoueur == 4 && (choixOrdi == 2 || choixOrdi == 5)) ||
                 (choixJoueur == 5 && (choixOrdi == 1 || choixOrdi == 3)))
        {
            printf("Vous gagnez cette manche !\n");
            scoreJoueur = scoreJoueur + 1;
        }
        else
        {
            printf("L'ordinateur gagne cette manche !\n");
            scoreOrdi = scoreOrdi + 1;
        }
        printf("Score actuel -> Vous : %d | Ordi : %d\n\n", scoreJoueur,
scoreOrdi);
        manche = manche + 1;
    }
    // Bilan de la partie
    printf("=== FIN DE LA PARTIE ===\n");
    printf("Score final -> Vous : %d | Ordi : %d\n", scoreJoueur, scoreOrdi);
    if (scoreJoueur > scoreOrdi)
    {
        printf("Bravo, vous avez gagné la partie !\n");
    }
    else if (scoreOrdi > scoreJoueur)
    {
        printf("L'ordinateur remporte la partie...\n");
    }
    else
    {
        printf("Match nul parfait !\n");
    }
    return 0;
}
```

Tester et sauvegarder (*valider/commit* puis *synchroniser/push* sur github).

Étape 2 - Procédure sans paramètre

Déplacer les instructions sous le commentaire `// Bilan de la partie` (hors `return 0`), dans une procédure (fonction `void`) `afficher_bilan` et remplacer ce commentaire par l'appel de la fonction.

Vérifier que le bilan s'affiche correctement.

Représenter le contenu de la fonction dans un logigramme exact fait sur Flow-C dont le `png` sera ajouté (glisser/déplacer) dans votre projet codespace.

Sauvegarder (*commit* puis *push* github).

Étape 3 - Procédure avec paramètres

Créer une procédure `afficher_choix` qui prend un entier en paramètre et affiche le choix correspondant. Exemple, si 1 est passé, la procédure affiche "Pierre".

Appeler cette fonction :

- pour afficher le choix de l'ordinateur autremant qu'avec un nombre
- dans une boucle `for` allant de 1 à 5 pour afficher le menu à l'utilisateur de manière plus évolutive.

Joindre au projet le logigramme Flow-C exact du code de ce nouvel affichage de menu.

Sauvegarder (*commit* puis *push* github).

Étape 4 - Fonctions

Faire les trois fonctions suivantes dont le code existe déjà dans le programme et les appeler en lieu et place du code existant :

- `partie_en_cours` qui retourne `true` si la partie doit continuer (moins de 7 manches et différence de score de moins de 2). Cette fonction prend le nombre de manches et les scores en paramètres.
- `saisie_joueur` sans paramètre qui retourne le choix du joueur après lui avoir affiché les options et répété l'action tant que la réponse était erronée. Le retour de la fonction est utilisé pour initialiser `choixJoueur`
- `joueur1_gagne` qui prend deux choix de joueur en paramètre et retourne `true` si le premier choix gagne.

Tester et sauvegarder (*commit* puis *push* github).

Remise

Le lien vers le dépôt github contenant le code et les png téléchargés depuis flowc nommés selon l'étape.